

Réplication MariaDB avec Galera et arbitre garbd

Architecture simplifiée du cluster Galera utilisé dans le TP

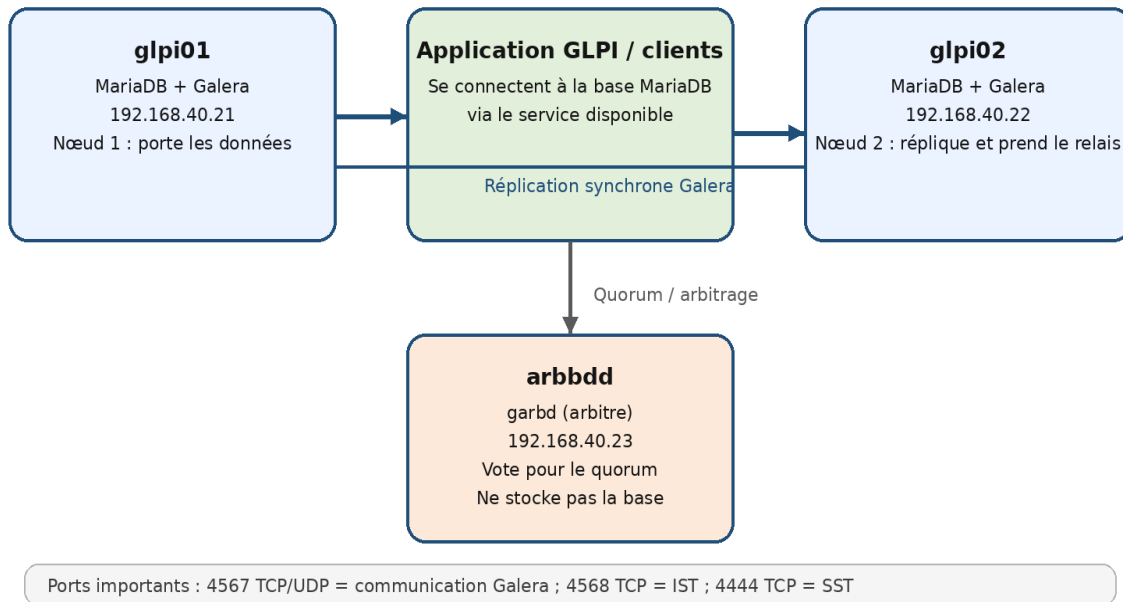


Figure 1 - Vue d'ensemble : deux nœuds MariaDB/Galera et un arbitre garbd.

Contexte	Infrastructure GLPI avec glpi01, glpi02 et arbbdd sur le réseau 192.168.40.0/24.
Public visé	Étudiants développeurs d'application qui veulent comprendre l'intérêt du cluster sans être spécialistes système.
Objectif pédagogique	Savoir lire l'architecture, configurer les composants, diagnostiquer les erreurs fréquentes et arrêter/démarrer proprement le cluster.

1. Pourquoi utiliser Galera dans ce contexte ?

Dans une application comme GLPI, la base de données contient les tickets, les utilisateurs, les catégories, les historiques et une grande partie de la valeur métier. Si un seul serveur MariaDB tombe, l'application peut devenir indisponible. Galera permet de garder plusieurs nœuds MariaDB synchronisés en quasi temps réel. L'arbitre garbd n'héberge pas la base mais aide le cluster à continuer à prendre une décision de quorum lorsqu'un nœud disparaît.

Idée simple à retenir

- glpi01 et glpi02 stockent les données et les répliquent.
- garbd ne contient pas la base ; il sert de témoin pour éviter un blocage du cluster.
- L'objectif n'est pas seulement la performance : c'est surtout la continuité de service et la cohérence des données.

2. Rôle de chaque composant

Composant	Rôle	Ce qu'il stocke	À quoi il faut faire attention
glpi01	Nœud MariaDB/Galera principal au démarrage du cluster.	Les bases MariaDB et l'historique Galera.	Nom de cluster, ports Galera, état wsrep, choix du bootstrap.
glpi02	Deuxième nœud MariaDB/Galera qui rejoint le cluster.	Les mêmes données que glpi01 après synchronisation.	Syntaxe du fichier 60-galera.cnf, ouverture des ports, état JOINER/JOINED/SYNCED.
arbbdd / garbd	Arbitre de quorum. Il vote mais ne stocke pas la base.	Aucune donnée applicative de la base.	Le bon fichier /etc/default/garb, le bon nom de groupe et les nœuds à contacter.
wsrep / Galera	Mécanisme de réplication et de cohérence entre les nœuds.	Métadonnées de réplication et état du cluster.	Variables wsrep cohérentes partout, lecture des statuts wsrep_cluster_size, wsrep_ready, wsrep_connected.

3. Les ports à connaître

Port	Rôle	Quand on le voit vraiment travailler
4567 TCP/UDP	Communication Galera entre les nœuds.	En permanence pour la vie du cluster.
4568 TCP	IST (Incremental State Transfer).	Quand un nœud rattrape un petit décalage.
4444 TCP	SST (State Snapshot Transfer).	Quand un nœud doit récupérer une copie plus complète de l'état.

Lecture simple des tests réseau

- Port 4567 ouvert : le dialogue Galera de base peut se faire.
- Ports 4568 et 4444 refusés au repos : ce n'est pas forcément anormal.
- Ces deux ports deviennent importants lorsqu'un transfert d'état est nécessaire.

4. Les fichiers de configuration utilisés

Le cœur du travail se trouve dans deux zones : le fichier 60-galera.cnf sur les nœuds MariaDB, et le fichier /etc/default/garb sur l'arbitre.

Machine	Fichier	Variables clés
glpi01	/etc/mysql/mariadb.conf.d/60-galera.cnf	wsrep_provider, wsrep_cluster_name, wsrep_cluster_address, wsrep_node_name, wsrep_node_address, wsrep_sst_method
glpi02	/etc/mysql/mariadb.conf.d/60-galera.cnf	Les mêmes variables, mais avec node_name et node_address du second nœud.
arbbdd	/etc/default/garb	GALERA_NODES, GALERA_GROUP, GALERA_PORT, GALERA_OPTIONS

```

GNU nano 8.4 /etc/mysql/mariadb.conf.d/60-galera.cnf
#
# * Galera-related settings
#
# See the examples of server wsrep.cnf files in /usr/share/mariadb
# and read more at https://mariadb.com/kb/en/galera-cluster/
[mysqld]
bind-address=0.0.0.0
[Galera]
wsrep_on=ON
wsrep_provider=/usr/lib/galera/libgalera_smm.so
wsrep_cluster_name="GLPI_CLUSTER"
wsrep_cluster_address="gcomm://192.168.40.21,192.168.40.22,192.168.40.23"
wsrep_node_name="glpi01"
wsrep_node_address="192.168.40.21"
# SST (simple pour TP)
wsrep_sst_method=rsync
# Requis Galera
binlog_format=ROW
default_storage_engine=InnoDB
innodb_autoinc_lock_mode=2

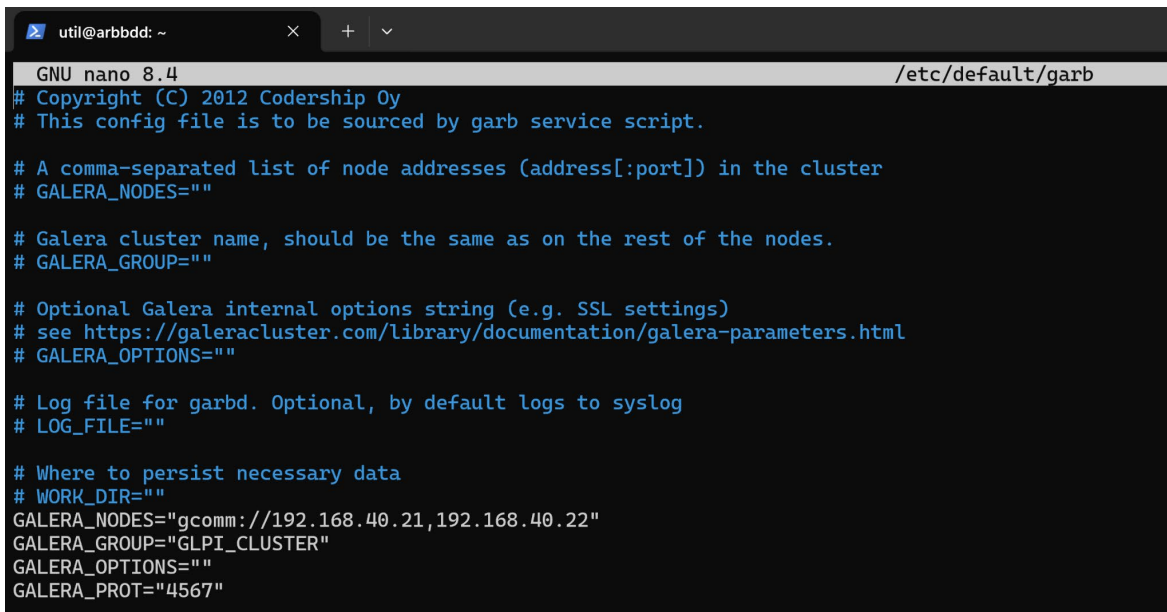
#[galera]
# Mandatory settings
#wsrep_on = ON
#wsrep_cluster_name = "MariaDB Galera Cluster"
#wsrep_cluster_address = gcomm://
#binlog_format = row
#default_storage_engine = InnoDB
#innodb_autoinc_lock_mode = 2

# Allow server to accept connections on all interfaces.
#bind-address = 0.0.0.0

# Optional settings
#wsrep_slave_threads = 1
#innodb_flush_log_at_trx_commit = 0

```

Figure 2 - Exemple de configuration Galera sur un nœud MariaDB.



```

util@arbbdd: ~
GNU nano 8.4 /etc/default/garb
# Copyright (C) 2012 Codership Oy
# This config file is to be sourced by garb service script.

# A comma-separated list of node addresses (address[:port]) in the cluster
# GALERA_NODES=""

# Galera cluster name, should be the same as on the rest of the nodes.
# GALERA_GROUP=""

# Optional Galera internal options string (e.g. SSL settings)
# see https://galeracluster.com/library/documentation/galera-parameters.html
# GALERA_OPTIONS=""

# Log file for garbd. Optional, by default logs to syslog
# LOG_FILE=""

# Where to persist necessary data
# WORK_DIR=""
GALERA_NODES="gcomm://192.168.40.21,192.168.40.22"
GALERA_GROUP="GLPI_CLUSTER"
GALERA_OPTIONS=""
GALERA_PROT="4567"

```

Figure 3 - Exemple de configuration de l'arbitre garbd.

5. Procédure complète de mise en œuvre

5.1 Configurer les deux nœuds MariaDB

- Sur glpi01, ouvrir /etc/mysql/mariadb.conf.d/60-galera.cnf et recopier une configuration de nœud MariaDB conforme à la **Figure 2**.
- Vérifier sur glpi01 les paramètres les plus sensibles : `wsrep_provider`, `wsrep_cluster_name="GLPI_CLUSTER"`, `wsrep_cluster_address="gcomm://192.168.40.21,192.168.40.22"`, `wsrep_node_name="glpi01"` et `wsrep_node_address="192.168.40.21"`.
- Sur glpi02, ouvrir le même fichier et reprendre la même structure que sur glpi01 (voir **Figure 2**), mais avec **`wsrep_node_name="glpi02"` et `wsrep_node_address="192.168.40.22"`**.
- Comparer ensuite les deux fichiers ligne par ligne : le nom du cluster et l'adresse gcomm doivent être identiques, alors que le nom du nœud et son adresse IP doivent être propres à chaque machine.

5.2 Configurer l'arbitre de quorum

- Sur arbbdd, installer le paquet `galera-arbitrator-4` puis ouvrir /etc/default/garb en vous appuyant sur la **Figure 3**.
- Renseigner `GALERA_NODES` avec les deux nœuds MariaDB, par exemple `192.168.40.21,192.168.40.22`, afin que l'arbitre sache quels membres contacter.
- Renseigner `GALERA_GROUP` avec le même nom logique que sur les nœuds MariaDB, ici `GLPI_CLUSTER` ; sinon l'arbitre observera un autre groupe et ne participera pas au bon quorum.
- Retenir le rôle de `garbd` : il ne stocke pas les bases de données de l'application ; il sert de vote supplémentaire pour aider le cluster à rester en composant Primary lorsqu'un des deux nœuds MariaDB disparaît.

5.3 Ouvrir les ports réseau sur glpi01 et glpi02

```
ufl allow 4567/tcp
ufl allow 4567/udp
ufl allow 4568/tcp
ufl allow 4444/tcp
ufl reload
```

5.4 Démarrer le cluster pas à pas sur chaque machine

- Étape A - Sur glpi01, lorsque tout le cluster est arrêté, lancer **galera_new_cluster**. Cette commande crée le composant primaire initial ; elle ne doit donc être utilisée que pour le tout premier démarrage ou après un arrêt total du cluster.
- Toujours sur glpi01, vérifier aussitôt le résultat avec **systemctl status mariadb --no-pager** puis **mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"** et **mysql -e "SHOW STATUS LIKE 'wsrep_cluster_status';"**. Après le bootstrap, on attend d'abord cluster_size = 1 et cluster_status = Primary.
- Étape B - Sur glpi02, lancer **systemctl start mariadb**. Le second nœud doit alors rejoindre glpi01. Pour suivre ce qu'il se passe, lire **journalctl -u mariadb -n 80 --no-pager** et repérer la progression JOINER, JOINED puis SYNCED, ainsi qu'un éventuel SST ou IST.
- Étape C - Quand les deux nœuds MariaDB sont opérationnels, lancer **systemctl start garb** sur arbbdd, puis vérifier **systemctl status garb**. L'arbitre complète le quorum mais n'est démarré qu'après les nœuds de données, afin d'éviter de diagnostiquer deux problèmes en même temps.

5.5 Vérifier immédiatement après le démarrage

- Sur glpi01, exécuter **mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"** puis **mysql -e "SHOW STATUS LIKE 'wsrep_cluster_status';"**. Quand glpi02 a rejoint le groupe, la taille doit passer à 2 et l'état doit rester Primary.
- Sur glpi02, exécuter **mysql -e "SHOW STATUS LIKE 'wsrep_connected';"** ; **mysql -e "SHOW STATUS LIKE 'wsrep_ready';"** ; **mysql -e "SHOW STATUS LIKE 'wsrep_local_state_comment';"**. Les valeurs attendues sont ON, ON et Synced.
- Sur arbbdd, vérifier l'arbitre avec **systemctl status garb** et, si besoin, **journalctl -u garb -n 30 --no-pager**. Le service doit être actif et utiliser le même nom de groupe que les deux nœuds MariaDB.

6. Quelques dysfonctionnements probables

Dans un TP, les problèmes arrivent souvent après la première tentative de démarrage. La logique est donc la suivante : d'abord appliquer la procédure, puis seulement ensuite diagnostiquer ce qui empêche un nœud de rejoindre le cluster.

1. **Étape 1 - Identifier le symptôme** : Un nœud affichait wsrep_cluster_size = 0 et wsrep_cluster_status = Disconnected. Cela signifie : "je suis lancé, mais je ne fais pas partie du cluster".
2. **Étape 2 - Vérifier la configuration** : Les fichiers ont été relus pour comparer le nom de cluster, les adresses, les noms de nœuds et la syntaxe des variables wsrep.
3. **Étape 3 - Corriger les fautes de frappe** : Dans un cluster, une seule faute sur wsrep_provider ou wsrep_node_name suffit à empêcher MariaDB de démarrer ou le nœud de rejoindre le groupe.

4. **Étape 4 - Contrôler les ports et le pare-feu** : Le service écoutait, mais UFW bloquait les ports 4567/4568/4444. Le ping fonctionnait, mais Galera ne pouvait pas dialoguer.
5. **Étape 5 - Relancer proprement le nœud** : Une fois la config et le pare-feu corrigés, glpi02 a reçu un SST puis un IST pour rejoindre glpi01.
6. **Étape 6 - Valider l'état final** : Les commandes wsrep montrent finalement un cluster de taille 2, en Primary, avec chaque nœud en état Synced.

```

root@glpi01:~# mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_cluster_size | 1     |
+-----+-----+
root@glpi01:~# mysql -e "SHOW STATUS LIKE 'wsrep_cluster_status';"
+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_cluster_status | Primary |
+-----+-----+
root@glpi01:~# mysql -e "SHOW STATUS LIKE 'wsrep_connected';"
+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_connected | ON    |
+-----+-----+
root@glpi01:~# mysql -e "SHOW STATUS LIKE 'wsrep_ready';"
+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_ready    | ON    |
+-----+-----+
root@glpi01:~# mysql -e "SHOW STATUS LIKE 'wsrep_local_state_comment';"
+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_local_state_comment | Synced |
+-----+-----+
root@glpi01:~# mysql -e "SHOW STATUS LIKE 'wsrep_local_state_comment';"
+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_local_state_comment | Synced |
+-----+-----+
root@glpi01:~# mysql -e "SHOW STATUS LIKE 'wsrep_cluster_name';"
root@glpi01:~#

```

Figure 4 - Lecture des statuts wsrep : premier nœud sain.

```

root@glpi02:~# mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
mysql -e "SHOW STATUS LIKE 'wsrep_cluster_status';"
mysql -e "SHOW STATUS LIKE 'wsrep_connected';"
mysql -e "SHOW STATUS LIKE 'wsrep_ready';"
mysql -e "SHOW STATUS LIKE 'wsrep_local_state_comment';"
mysql -e "SHOW STATUS LIKE 'wsrep_cluster_name';"
+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_cluster_size | 0 |
+-----+-----+

+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_cluster_status | Disconnected |
+-----+-----+

+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_connected | OFF |
+-----+-----+

+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_ready | OFF |
+-----+-----+
root@glpi02:~#

```

Figure 5 - Lecture des statuts wsrep : nœud déconnecté avant correction.

```

root@glpi01:~# ss -lntup | grep -E '4567|4568|4444'
tcp LISTEN 0 4096 0.0.0.0:4567 0.0.0.0:* users:((("mariadb",pid=2481,fd=8))

root@glpi01:~# ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp (OpenSSH) ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
22/tcp (OpenSSH (v6)) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)

root@glpi01:~#

```

Figure 6 - Pare-feu UFW : ports Galera à autoriser sur glpi01.

7. Comment lire les messages Galera sans être expert système

Message / état	Interprétation simple
Disconnected	Le nœud est lancé mais il ne fait pas partie du cluster.
Primary	Le cluster a assez de membres pour fonctionner correctement.
JOINER	Le nœud reçoit une synchronisation.
JOINED	Le nœud est presque prêt mais doit encore terminer sa mise à jour.
SYNCED	Le nœud est intégré et prêt à répondre aux connexions.
SST	Le nœud reçoit un transfert d'état complet ou quasi complet.
IST	Le nœud récupère seulement les transactions manquantes.
Conseil pour développeur • Quand vous voyez <code>wsrep_cluster_size = 2</code>, pensez : “mes deux serveurs de base sont dans le groupe”. • Quand vous voyez <code>wsrep_local_state_comment = Synced</code>, pensez : “ce nœud est exploitable”. • Quand vous voyez <code>Primary</code>, pensez : “le cluster a un quorum valide”.	

8. Diagnostic guidé à reproduire en cas de problème

- 1. Vérifier si MariaDB démarre : `systemctl status mariadb --no-pager`
- 2. Lire le journal : `journalctl -u mariadb -n 80 --no-pager`
- 3. Vérifier la cohérence des variables `wsrep` : `grep -R "wsrep_cluster_name" /etc/mysql 2>/dev/null`
- 4. Tester le port principal : `nc -zvn -w 3 192.168.40.21 4567`
- 5. Contrôler l'écoute locale : `ss -lntp | grep 4567`
- 6. Vérifier le pare-feu : `ufw status verbose`
- 7. Contrôler l'état final du cluster avec les variables `wsrep`.

```

root@glpi01:~# grep -R "wsrep_cluster_name" /etc/mysql /etc/mysql/mariadb.conf.d 2>/dev/null
/etc/mysql/mariadb.conf.d/60-galera.cnf:wsrep_cluster_name="GLPI_CLUSTER"
/etc/mysql/mariadb.conf.d/60-galera.cnf:#wsrep_cluster_name      = "MariaDB Galera Cluster"
/etc/mysql/mariadb.conf.d/60-galera.cnf:wsrep_cluster_name="GLPI_CLUSTER"
/etc/mysql/mariadb.conf.d/60-galera.cnf:#wsrep_cluster_name      = "MariaDB Galera Cluster"
root@glpi01:~# nano /etc/mysql/mariadb.conf.d/60-galera.cnf
root@glpi01:~# ss -lnup | grep 4567
ss -lntp | grep 4567
LISTEN 0      4096          0.0.0.0:4567      0.0.0.0:*        users:((("mariabdb",pid=2481,fd=8))

root@glpi01:~# ss -lnup | grep 4567
root@glpi01:~# ss -lntp | grep 4567
LISTEN 0      4096          0.0.0.0:4567      0.0.0.0:*        users:((("mariabdb",pid=2481,fd=8))

root@glpi01:~#

```

Figure 7 - Vérification des noms de cluster et des ports en écoute.


```

root@glpi02:~# nano /etc/mysql/mariadb.conf.d/60-galera.cnf
root@glpi02:~# systemctl restart mariadb
root@glpi02:~# grep -R "wsrep_cluster_name" /etc/mysql /etc/mysql/mariadb.conf.d 2>/dev/null
/etc/mysql/mariadb.conf.d/60-galera.cnf:wsrep_cluster_name = "GLPI_CLUSTER"
/etc/mysql/mariadb.conf.d/60-galera.cnf:wsrep_cluster_name = "GLPI_CLUSTER"
root@glpi02:~# nano /etc/mysql/mariadb.conf.d/60-galera.cnf
root@glpi02:~# nc -zvn -w 3 192.168.40.21 4567.168.40.21 4567
nc -zvn -w 3 192.168.40.21 4568
nc -zvn -w 3 192.168.40.21 4444
(UNKNOWN) [192.168.40.21] 4567 (?) open
(UNKNOWN) [192.168.40.21] 4568 (?) : Connection refused
(UNKNOWN) [192.168.40.21] 4444 (?) : Connection refused
root@glpi02:~# nc -zvn -w 3 192.168.40.21 4567
(UNKNOWN) [192.168.40.21] 4567 (?) open
root@glpi02:~# nc -zvn -w 3 192.168.40.21 4568
(UNKNOWN) [192.168.40.21] 4568 (?) : Connection refused
root@glpi02:~# nc -zvn -w 3 192.168.40.21 4444
(UNKNOWN) [192.168.40.21] 4444 (?) : Connection refused
root@glpi02:~#

```

Figure 8 - Tests réseau depuis glpi02 vers glpi01.

9. Arrêter et démarrer proprement le cluster

9.1 Arrêt planifié

- Arrêter d'abord glpi02 : `systemctl stop mariadb`
- Puis arrêter glpi01 : `systemctl stop mariadb`
- Enfin arrêter l'arbitre : `systemctl stop garb`

9.2 Redémarrage complet après extinction totale

- Sur le nœud qui possède `safe_to_bootstrap: 1` dans `grastate.dat` avec `cat /var/lib/mysql/grastate.dat`, lancer `galera_new_cluster`.
- Vérifier que ce premier nœud a `wsrep_cluster_size = 1` et `wsrep_cluster_status = Primary`.
- Démarrer le second nœud avec `systemctl start mariadb`.
- Contrôler qu'il passe de `JOINER` à `JOINED` puis à `SYNCED`.
- Démarrer ensuite l'arbitre avec `systemctl start garb`.

Règle importante

- On n'utilise `galera_new_cluster` que lorsque tout le cluster est arrêté.
- Si un nœud du cluster est déjà actif, on démarre les autres avec `systemctl start mariadb`.
- Un mauvais bootstrap peut créer un groupe isolé et compliquer la reprise.

10. Validation finale du TP

Contrôle	Commande	Résultat attendu
Le cluster contient bien 2 nœuds	<code>SHOW STATUS LIKE "wsrep_cluster_size";</code>	Valeur = 2 sur glpi01 et glpi02
Le cluster est valide	<code>SHOW STATUS LIKE "wsrep_cluster_status";</code>	Valeur = Primary
Le nœud est prêt	<code>SHOW STATUS LIKE "wsrep_local_state_comment";</code>	Valeur = Synced
La réplication fonctionne	<code>CREATE DATABASE test_galera;</code> <code>puis SHOW DATABASES LIKE</code>	La base créée sur un nœud apparaît sur l'autre

	"test_galera";	
--	-----------------------	--

```

root@glpi02:~# systemctl status mariadb --no-pager
● mariadb.service - MariaDB 11.8.3 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-03-30 16:36:48 CEST; 13min ago
  Invocation: 795d2479f140486d90d801875f94d9ff
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 3761 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d /var/run/mysqld (code=exited, status=0/SUCCESS)
   Process: 3763 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && VAR= [ VAR=/usr/bin/galera_recovery ]; [ $? -eq 0 ] && echo _WSREP_START_POSITI
ON=$VAR > /run/mysqld/wsrep-start-position || exit 1 (code=exited, status=0/SUCCESS)
   Process: 4079 ExecStartPost=/bin/rm -f /run/mysqld/wsrep-start-position (code=exited, status=0/SUCCESS)
   Process: 4081 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
  Main PID: 3834 (mariadbd)
    Status: "WSREP state transfer (role joiner) completed."
      Tasks: 14 (limit: 14082)
     Memory: 387.2M (peak: 392.1M)
        CPU: 7.415s
    CGroup: /system.slice/mariadb.service
            └─3834 /usr/sbin/mariadbd --wsrep_start_position=00000000-0000-0000-0000-000000000000:-1

mars 30 16:36:48 glpi02 mariadbd[3834]: 2026-03-30 16:36:48 0 [Note] WSREP: Processing event queue:... 100.0% (1/1 events) complete.
mars 30 16:36:48 glpi02 mariadbd[3834]: 2026-03-30 16:36:48 0 [Note] WSREP: Shifting JOINED -> SYNCED (TO: 2)
mars 30 16:36:48 glpi02 mariadbd[3834]: 2026-03-30 16:36:48 2 [Note] WSREP: Server glpi02 synced with group
mars 30 16:36:48 glpi02 mariadbd[3834]: 2026-03-30 16:36:48 2 [Note] WSREP: Server status change joined -> synced
mars 30 16:36:48 glpi02 mariadbd[3834]: 2026-03-30 16:36:48 2 [Note] WSREP: Synchronized with group, ready for connections
mars 30 16:36:48 glpi02 mariadbd[3834]: 2026-03-30 16:36:48 2 [Note] WSREP: wsrep_notify_cmd is not defined, skipping notification.
mars 30 16:36:48 glpi02 systemd[1]: Started mariadb.service - MariaDB 11.8.3 database server.
mars 30 16:36:48 glpi02 /etc/mysql/debian-start[4083]: Upgrading MariaDB tables if necessary.
mars 30 16:36:48 glpi02 /etc/mysql/debian-start[4094]: Checking for insecure root accounts.
mars 30 16:36:48 glpi02 /etc/mysql/debian-start[4098]: Triggering myisam-recover for all MYISAM tables and aria-recover for all Aria tables

```

Figure 9 - Statut du service MariaDB et fin de synchronisation sur glpi02.

```

root@glpi02:~# mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
mysql -e "SHOW STATUS LIKE 'wsrep_cluster_status';"
mysql -e "SHOW STATUS LIKE 'wsrep_connected';"
mysql -e "SHOW STATUS LIKE 'wsrep_ready';"
mysql -e "SHOW STATUS LIKE 'wsrep_local_state_comment';"
+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_cluster_size | 2 |
+-----+-----+

+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_cluster_status | Primary |
+-----+-----+

+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_connected | ON |
+-----+-----+

+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_ready | ON |
+-----+-----+

+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_local_state_comment | Synced |
+-----+-----+
root@glpi02:~#

```

Figure 10 - Variables wsrep finales sur glpi02 : nœud synchronisé.

```

root@glpi01:~# mysql -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
mysql -e "SHOW STATUS LIKE 'wsrep_cluster_status';"
+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_cluster_size | 2 |
+-----+-----+
+-----+-----+
| Variable_name | Value |
+-----+-----+
| wsrep_cluster_status | Primary |
+-----+-----+
root@glpi01:~# |

```

Figure 11 - Variables wsrep finales sur glpi01 : cluster à 2 nœuds.

```

root@glpi02:~# mysql -e "SHOW DATABASES LIKE 'test_galera';"
+-----+
| Database (test_galera) |
+-----+
| test_galera |
+-----+
root@glpi02:~#

```

Figure 12 - Test applicatif simple : création de la base test_galera sur glpi01.

```

root@glpi01:~# mysql -e "CREATE DATABASE test_galera;"
root@glpi01:~# |

```

Figure 13 - Vérification sur glpi02 : la base test_galera est bien répliquée.

11. Conclusion

L'idée essentielle est la suivante : un cluster Galera n'est pas une "boîte magique". Il fonctionne si, et seulement si, trois dimensions restent cohérentes : la configuration, le réseau et l'état du cluster. Dans ce TP, une faute de frappe, un nom de cluster incohérent ou un pare-feu trop strict peuvent suffire à empêcher la réplication. Une fois la cohérence rétablie, les indicateurs wsrep racontent clairement l'histoire du cluster.

À retenir

- Même nom de cluster partout.
- Même groupe de nœuds à joindre.
- Ports Galera ouverts sur les deux nœuds.
- Lecture systématique des états wsrep pour confirmer la santé du cluster.
- Bootstrap uniquement quand tout le cluster est à l'arrêt.